

Progress Report Appendix — SNN Temporal Multiplexing

Section 1 — Step 1: Single-Query Baseline

1.1 Experimental Setup

- **Task:** Learn one of 8 binary boolean operations (AND, OR, NAND, NOR, XOR, XNOR, A_IMP_B, B_IMP_A) from noisy Poisson spike inputs.
- **Network:** Input (2 neurons) → Delayed Synaptic Layer → LIF Hidden Layer (h neurons) → Linear Readout
- **Training modes compared:**
 - **weights_only** — synaptic delays fixed at $d = 0$; only weights trained
 - **delays_only** — weights frozen after random init; only delays trained
 - **weights_and_delays** — both weights and delays trained jointly
- **Sweep:** 8 ops × 3 modes × $h \in \{10, 20, 50\}$, 200 epochs, seed = 42, GPU
- **Encoding:** Rate coding — $A = 1 \rightarrow r_{\text{on}} = 400 \text{ Hz}$, $A = 0 \rightarrow r_{\text{off}} = 10 \text{ Hz}$; $dt = 1 \text{ ms}$, $T = 50 \text{ ms}$

1.2 Results by Training Mode

Training Mode	Mean Acc	Min	Max	Runs $\geq 95\%$
weights_only	0.878	0.769	0.940	0 / 24
delays_only	0.737	0.517	0.892	0 / 24
weights_and_delays	0.954	0.846	0.989	15 / 24

1.3 Accuracy by Mode × Hidden Size (mean over all ops)

Hidden Size	weights_only	delays_only	weights_and_delays
h = 10	0.853	0.706	0.924
h = 20	0.888	0.753	0.962
h = 50	0.893	0.753	0.974

1.4 Per-Op Accuracy at h = 50 (**weights_and_delays**)

Operation	Accuracy	$\geq 95\%$
AND	0.988	PASS
OR	0.986	PASS
NAND	0.989	PASS
NOR	0.985	PASS

Operation	Accuracy	≥ 95%?
A_IMP_B	0.985	PASS
B_IMP_A	0.978	PASS
XNOR	0.952	PASS
XOR	0.932	FAIL

1.5 Energy — Mean Hidden Spikes/Trial at h = 50

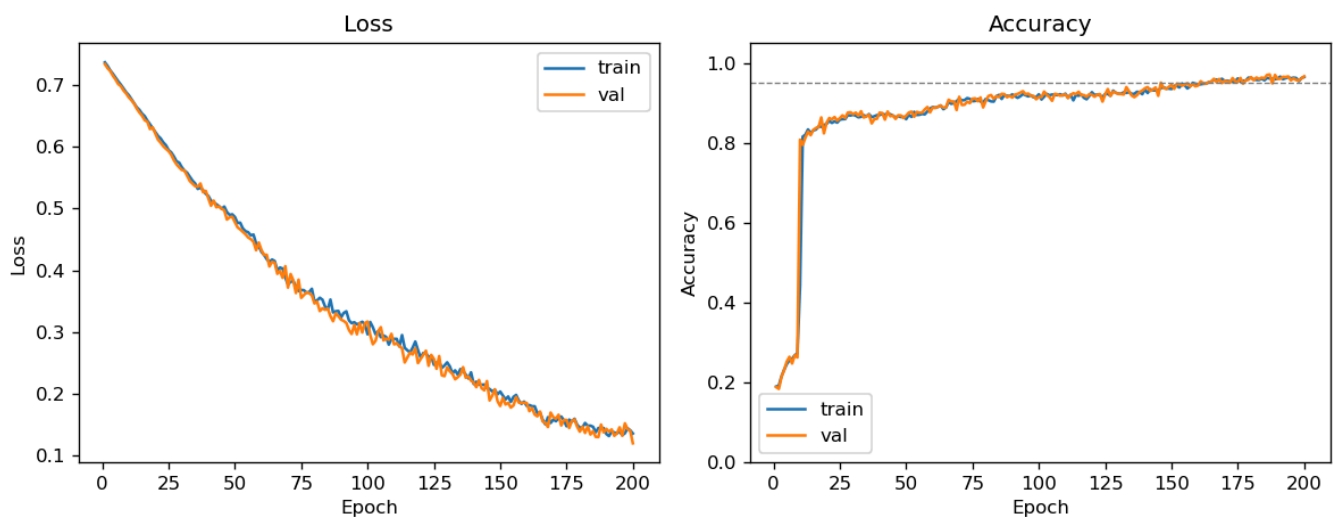
Training Mode	Spikes / Trial
<code>weights_only</code>	34.1
<code>weights_and_delays</code>	27.7
<code>delays_only</code>	2.5

1.6 Key Findings

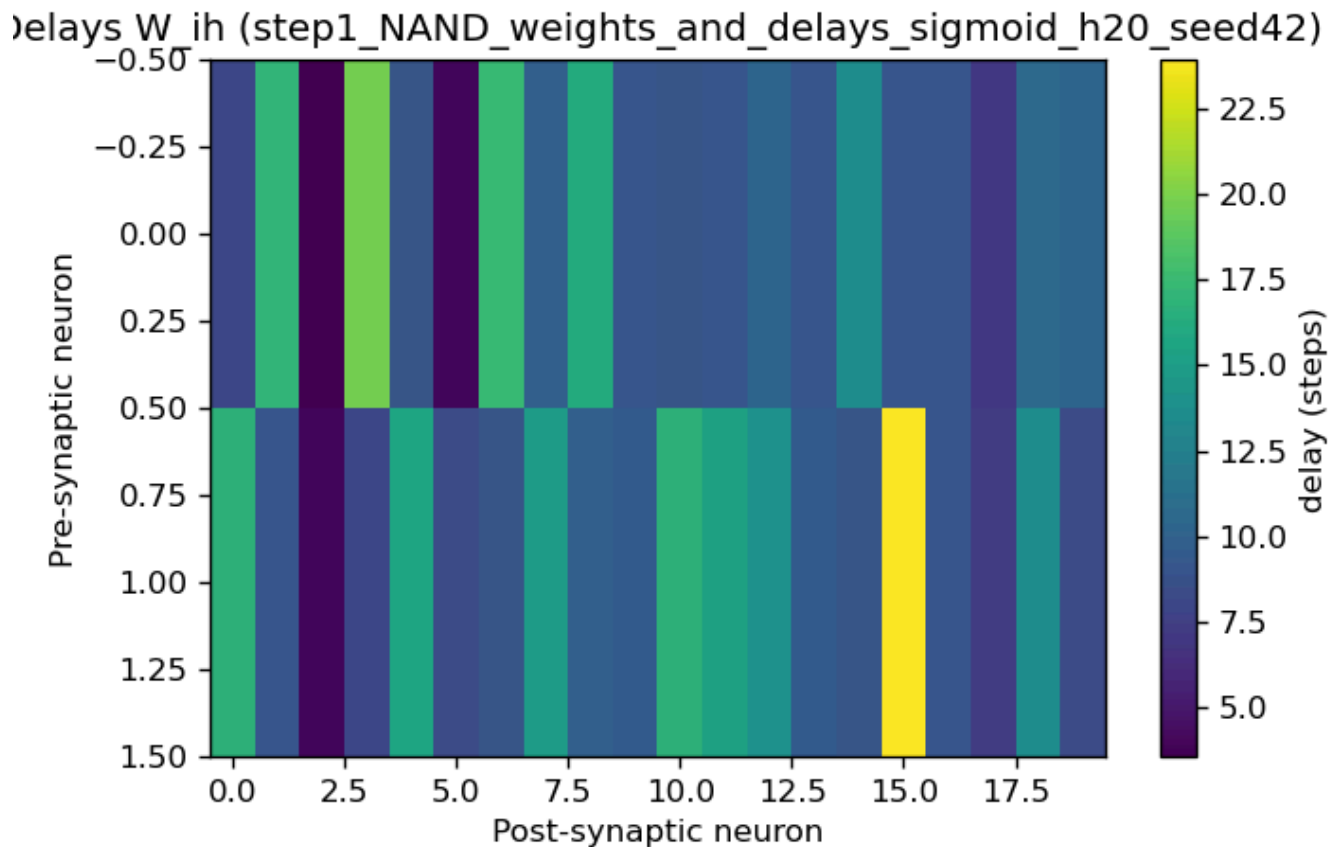
1. `weights_and_delays` outperforms `weights_only` by +7–8% at every hidden size; the gap widens slightly with larger networks.
2. `delays_only` collapses on XOR/XNOR (mean accuracy ~0.58, near chance), while performing reasonably on simple ops (AND/OR/NAND/NOR).
3. No mode achieves ≥95% accuracy without joint weight+delay training.
4. XOR is the hardest operation: even the best condition (h = 50, `weights_and_delays`) only reaches 93.2%, below the 95% threshold. NAND is the most stable operation across all modes and is chosen as the focal operation for Step 2.
5. `weights_and_delays` uses ~19% fewer spikes than `weights_only` while achieving higher accuracy, suggesting delays reduce redundant firing.

1.7 Representative Figures

Training curves — NAND, h = 20, `weights_and_delays`:



Learned delay distribution — NAND, h = 20, `weights_and_delays`:



Section 2 — Step 2 Plan A: Serial Time-Slot Multiplexing

2.1 Motivation

Step 1 established that trainable delays improve single-query accuracy. The next question is whether they also enable *temporal multiplexing* — processing more queries (larger K) within a fixed neuron budget while maintaining accuracy. Plan A is the most natural instantiation of this idea: divide one trial into K sequential time slots, inject one query per slot, and read out each slot independently.

2.2 Design

```

Trial structure (K = 4 example):
|-- slot 0 --|-- slot 1 --|-- slot 2 --|-- slot 3 --|
| win | rd |g | win | rd |g | win | rd |g | win | rd |g |
  20   10 5   20   10 5   20   10 5   20   10 5

win = input injection window (20 steps)
rd  = readout window (10 steps)
g   = gap / silence (5 steps)
slot_len = 35 steps; T = K × 35 steps

```

- **Network:** Input (2 neurons) → Delayed Synaptic Layer → LIF Hidden ($h = 20$) → per-slot Linear Readout (1 output per slot)

- **Training modes:** `weights_and_delays` (trainable delays, sigmoid parameterization) vs. `weights_only_d0` (fixed $d = 0$)
- **Sweep:** $K = 1$ to 20, NAND, $h = 20$, seed = 42, 100 epochs

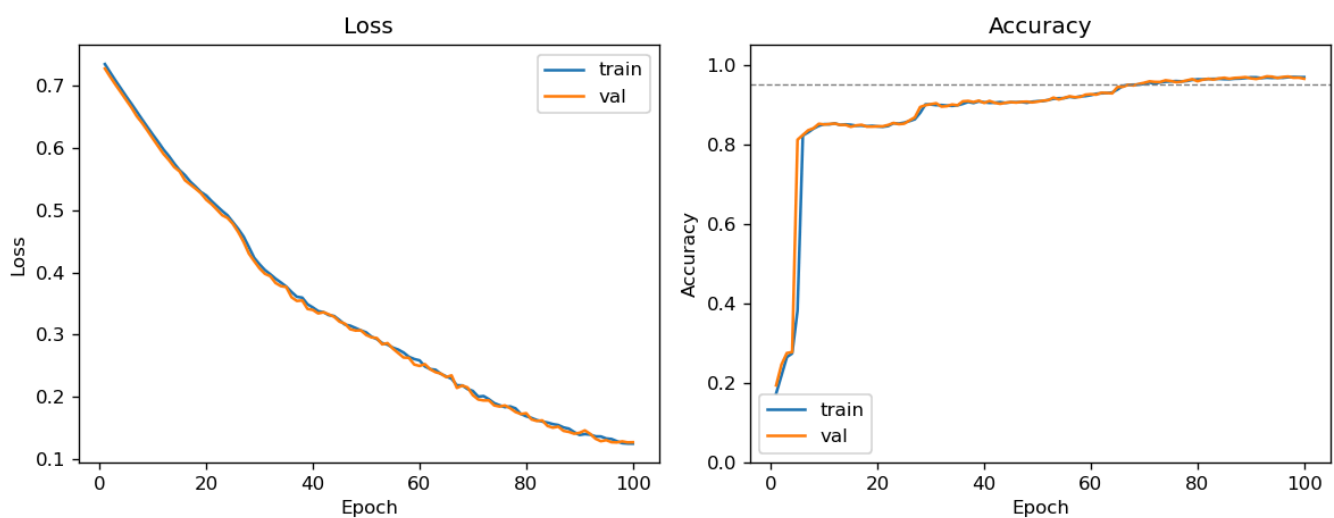
2.3 Results

K	<code>weights_only_d0</code>	<code>w_and_d_continuous</code>
1	0.797	0.951 ✓
2	0.784	0.957 ✓
4	0.796	0.971 ✓
6	0.795	0.967 ✓
8	0.794	0.971 ✓
10	0.807	0.970 ✓
12	0.804	0.966 ✓
14	—	0.972 ✓
16	—	0.972 ✓
18	—	0.970 ✓
20	—	0.973 ✓

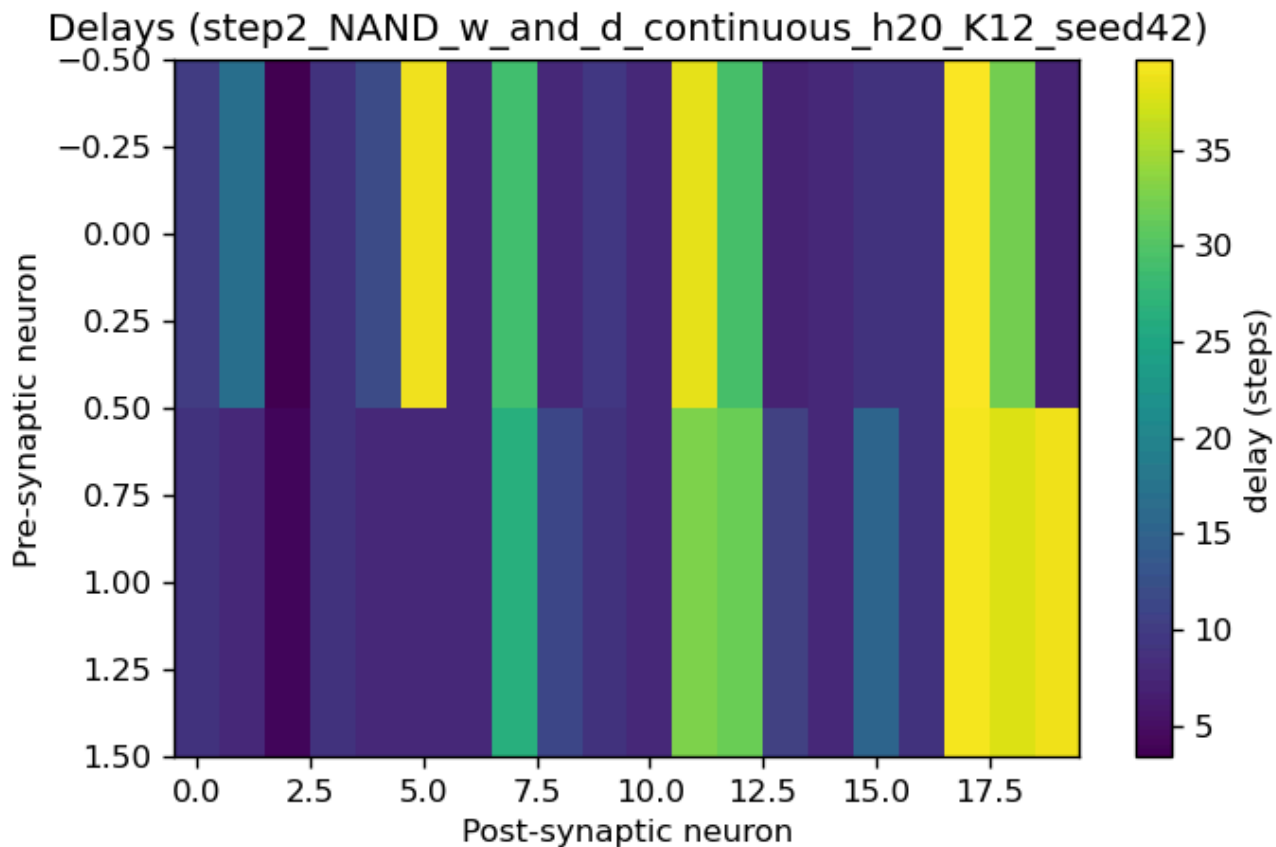
Max K @ 95% threshold: `weights_only_d0` = 0 (never reached); `w_and_d_continuous` = ≥ 20 (no degradation observed)

2.3 Representative Figures

Training curves — NAND, $K = 12$, `w_and_d_continuous`:



Learned delay distribution — NAND, $K = 12$, `w_and_d_continuous`:



Section 3 — Step 2 Plan C: Simultaneous Multi-Query (2K Channels)

3.1 Motivation

Plan A's slot isolation prevents genuine resource competition between queries. Plan C eliminates this by injecting all K queries simultaneously over a fixed trial length $T = 30$ steps (independent of K), giving each query its own pair of dedicated input channels (A_k, B_k). All $2K$ channels activate at the same time, forcing the 20 shared hidden neurons to process K concurrent queries within the same temporal window.

3.2 Design

```
Input channels: [A_0, B_0, A_1, B_1, ..., A_{K-1}, B_{K-1}] (2K total)
Injection:      t = 0 ... win_len (all 2K channels simultaneously)
Silence:       t = win_len ... T (readout accumulation)
T = 30 steps (fixed, regardless of K)

Network: 2K inputs → DelayedSynapticLayer(2K → 20) → LIF Hidden (20 neurons)
          → readout accumulation → Linear(20, K) → K output logits
```

Three conditions were compared:

- **A: `w_and_d_continuous`** — trainable weights + trainable delays (sigmoid)
- **B: `weights_only_d20`** — trainable weights, fixed $d = 20$ for all synapses
- **C: `weights_only_d0`** — trainable weights, fixed $d = 0$

Condition B was added as a controlled baseline to isolate the alignment effect (B vs. C) from the capacity effect (A vs. B).

3.3 Results — Accuracy

K	A: w_and_d	B: w_only_d20	C: w_only_d0	A–B (capacity)	B–C (alignment)
1	0.9730	0.9570	0.8010	+0.016	+0.156
2	0.9515	0.9285	0.7735	+0.023	+0.155
3	0.9433	0.9050	0.7720	+0.038	+0.133
4	0.9315	0.8942	0.7697	+0.037	+0.125
5	0.9066	0.8722	0.7660	+0.034	+0.106
6	0.8988	0.8662	0.7692	+0.033	+0.097
8	0.8863	0.8646	0.7598	+0.022	+0.105
10	0.8835	0.8514	0.7616	+0.032	+0.090
12	0.8663	0.8551	0.7587	+0.011	+0.096

Effect decomposition:

- Alignment effect (B – C): mean **+11.8%**, range 9.0%–15.6%
- Capacity effect (A – B): mean **+2.7%**, range 1.1%–3.8%
- Alignment / Capacity ratio: **4.3×**

Max K @ accuracy thresholds:

Condition	Max K @ 95%	Max K @ 90%
A: w_and_d_continuous	K = 2	K = 5
B: weights_only_d20	K = 1	K = 3
C: weights_only_d0	0	0

3.4 Results — Energy Efficiency (K / total hidden spikes)

K	A: w_and_d	B: w_only_d20	C: w_only_d0
1	0.244	0.444	0.109
2	0.502	0.681	0.236
4	0.401	0.840	0.216
8	0.339	0.590	0.343
12	0.378	0.726	0.283

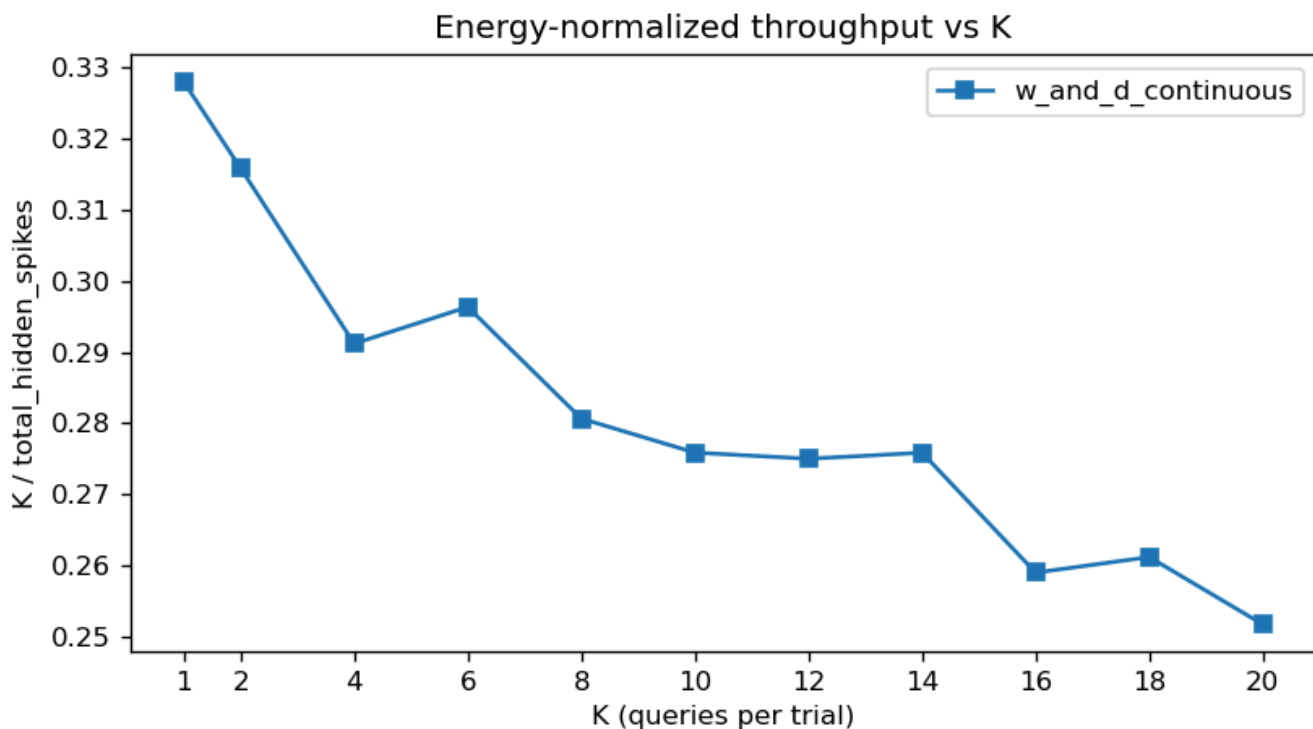
Higher K/spk means more queries completed per spike (better energy efficiency).

3.5 Key Findings

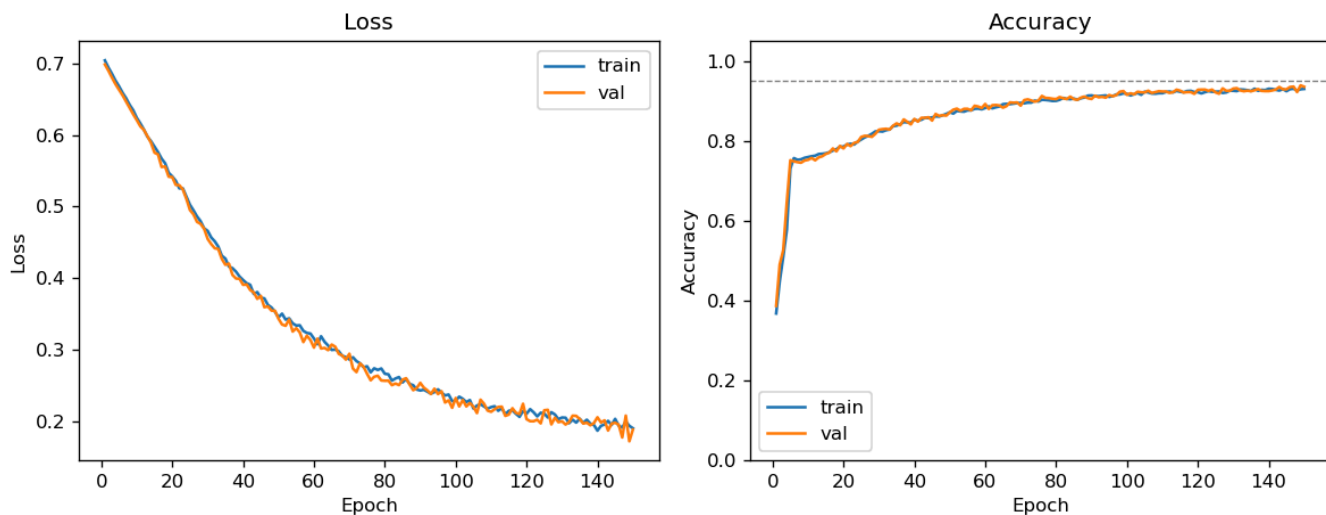
1. **Alignment dominates (4.3×):** The primary benefit of delays is to shift input spikes from $t \approx 0$ to arrive at $t \approx \text{win_len}$, aligning them with the readout window. This structural effect is independent of K .
2. **Trainable delays add modest capacity (+2.7%):** Compared to a well-chosen fixed delay ($d = 20$), trainable delays provide a real but small accuracy gain. Importantly, this gain does not grow monotonically with K (peak at $K = 3$), suggesting no true temporal routing capacity scaling in this design.

3.6 Representative Figures

K vs. energy throughput (K/spk):



Training curves — NAND, K = 4, w_and_d_continuous (Plan C):



Section 4 — Step 2 Plan D: Sequential Shared-Channel Injection

4.1 Motivation

Plan C showed that dedicated per-query input channels allow weights to separate queries spatially, making delays secondary. Plan D removes this escape route by using only 2 shared input channels for all K queries, with each query injected in a different time sub-window.

4.2 Design

Input channels: [A, B] (2 only, shared by all K queries)
 Query k is injected in: $t \in [k \times \text{sub_win}, (k+1) \times \text{sub_win})$
 $\text{sub_win} = \text{win_len} // K$ (e.g., $K=4$: $\text{sub_win}=5$ steps)
 $T = \text{win_len} + \text{read_len} = 30$ steps (fixed, regardless of K)

Network: 2 inputs $\rightarrow$ DelayedSynapticLayer($2 \rightarrow 20$) $\rightarrow$ LIF Hidden (20)
 $\rightarrow$ readout accumulation $\rightarrow$ Linear(20, K) $\rightarrow$ K output logits

The structural difference from Plan C:

- **Plan C:** Weights $W[A_k, j]$ can be set independently for each query $k \rightarrow$ spatial separation
- **Plan D:** Only $W[A, j]$ exists $\rightarrow$ same weight response to channel A at every time step $\rightarrow$ spatial separation impossible

4.3 Results

K	sub_win	w_and_d	d0	Gap (w-d0)	w_and_d K/spk	d0 K/spk
1	20	98.5%	79.2%	+19.3%	0.212	0.113
2	10	91.2%	76.4%	+14.8%	0.333	0.247
3	6	82.5%	76.8%	+5.7%	0.540	0.406
4	5	79.2%	76.5%	+2.7%	0.516	0.855
5	4	78.0%	76.1%	+1.9%	0.776	0.463

Max K @ 95%: w_and_d = $K = 1$; **Max K @ 90%:** w_and_d = $K = 2$

4.4 Plan D vs. Plan C Comparison (same K)

K	Plan C gap (2K channels)	Plan D gap (2 channels)	Interpretation
2	+17.8%	+14.8%	Comparable — sub_win = 10 provides sufficient signal
3	+17.1%	+5.7%	Plan D collapses — sub_win = 6 causes signal sparsity

4.5 Interpretation

Why the gap collapses at $K \geq 3$ (signal sparsity):

K	sub_win	Expected spikes/query
2	10	$10 \times 0.4 = 4.0$ ✓ (sufficient)
3	6	$6 \times 0.4 = 2.4$ (noisy)
4	5	$5 \times 0.4 = 2.0$ (very noisy)
5	4	$4 \times 0.4 = 1.6$ (learning barely possible)

Both conditions converge to the NAND label prior ($\sim 75\%$) because neither can extract reliable information from 2–4 spikes per query. The gap narrows not because delays lose capacity, but because **w_and_d** degrades faster than **d0**'s prior-based baseline.

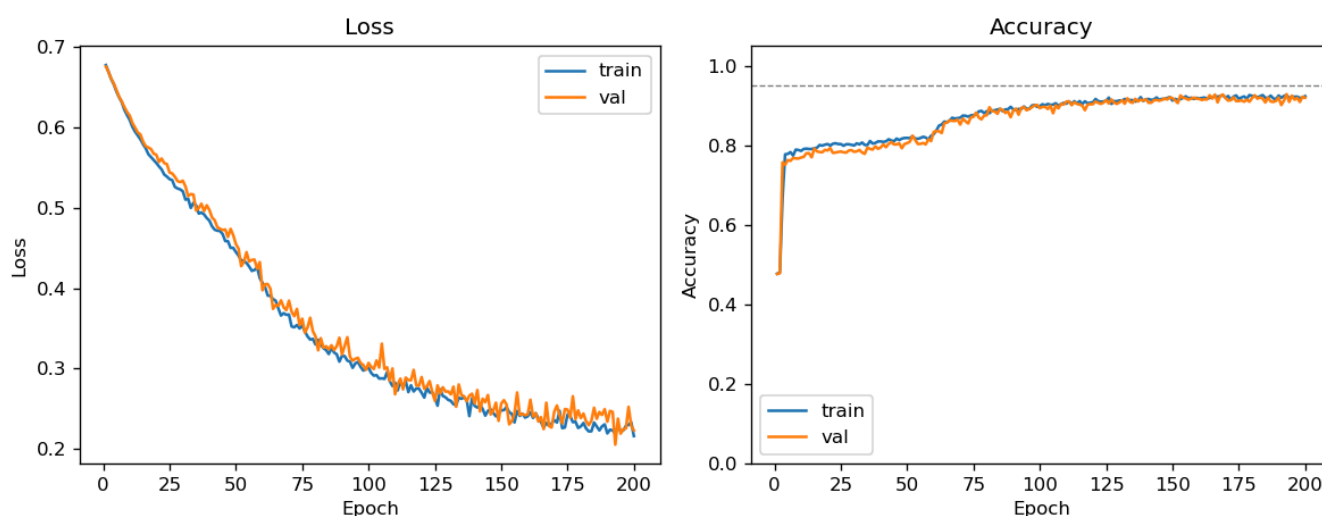
Proposed fix: Extend **win_len** from 20 to 60 steps $\rightarrow$ $K = 3$ gets **sub_win** = 20 steps, matching $K = 1$'s current signal density. This is the proposed next experiment.

4.6 Structural Significance

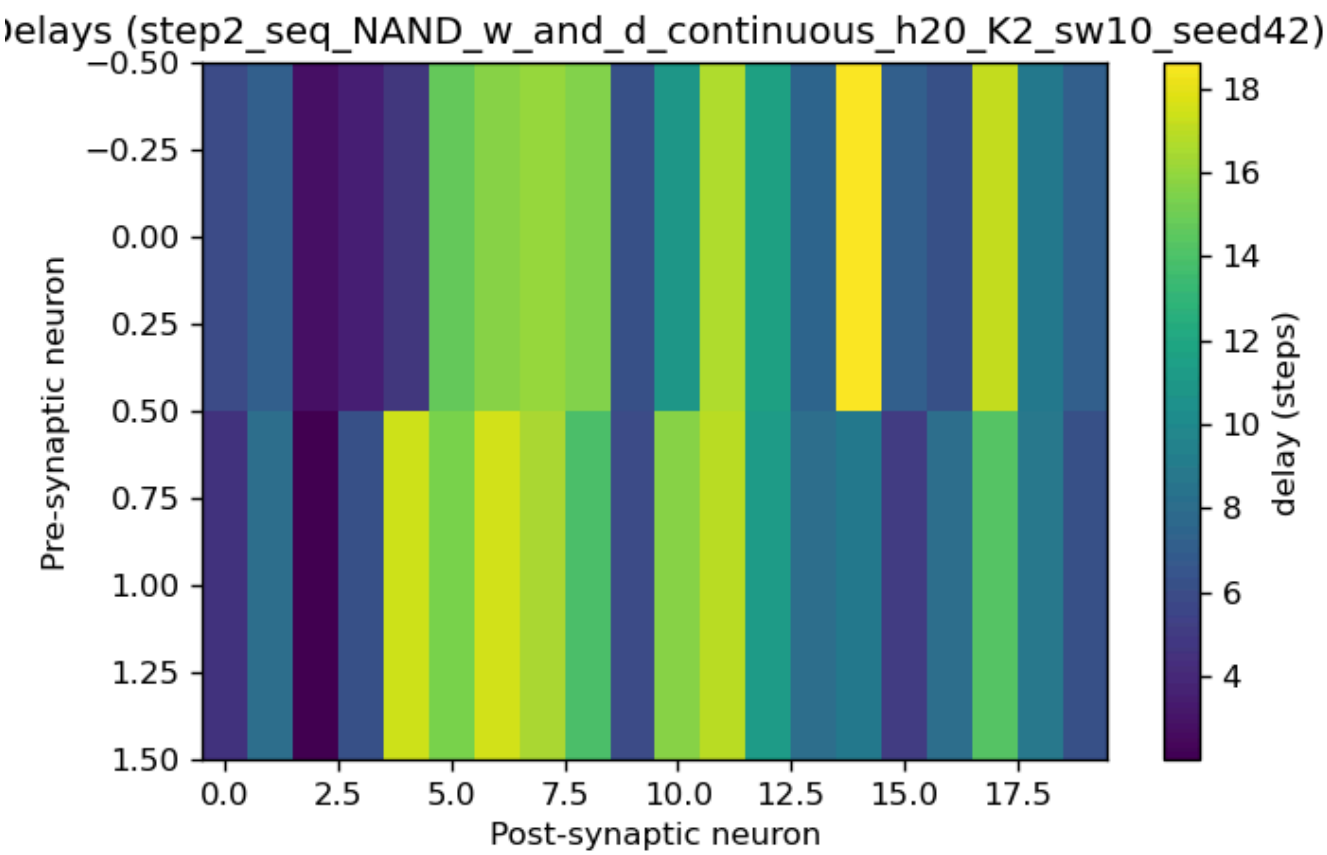
- The $K = 1$ result (+19.3%) is structurally analogous to Plan A and Plan C (alignment), confirming consistency across designs.
- The $K = 2$ result (+14.8%) is the most direct demonstration of temporal routing in the project: with only 2 shared channels, trainable delays enable the network to distinguish which of the two injection sub-windows a query came from, while fixed $d = 0$ cannot do so even in principle.

4.7 Representative Figures

Training curves — NAND, $K = 2$, **sub_win** = 10, **w_and_d** (Plan D):



Learned delay distribution — NAND, $K = 2$, **sub_win** = 10, **w_and_d** (Plan D):



Section 5 — Unified Interpretation

5.1 Three Designs Summary

Experiment	Core Design	Role of Delays	Key Result
Plan A	K serial time slots, 2 channels	Time alignment — shift input spikes into the readout window	K = 1–20 all ≥95%; no true multiplexing (LIF isolation makes each slot independent)
Plan C	K simultaneous queries, 2K channels	Alignment dominant (+11.8%) + small capacity (+2.7%)	Weights handle spatial separation; delays are secondary
Plan D	K sequential queries, 2 shared channels	Temporal routing — structurally necessary for query separation	K = 2 cleanly demonstrated (+14.8%); K ≥ 3 limited by signal sparsity

5.2 The Core Thesis Question

Are trainable synaptic delays a necessary mechanism for temporal multiplexing, or merely a useful one?

The three designs give a nuanced answer:

- Plan A:** Delays are *necessary* for accurate slot-structured processing (d = 0 fails even at K = 1), but not for *multiplexing* — the slot structure handles separation.

- **Plan C:** Delays are *useful* but not *necessary* — a fixed $d = 20$ achieves close performance because weights can separate queries spatially.
- **Plan D:** Delays are **necessary** — fixed $d = 0$ is structurally incapable of distinguishing temporally-offset queries sharing the same input channels. This is the most defensible evidence for delays as a fundamental computational mechanism.